

UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

SISTEM DE DETECTARE A INTRUZIUNILOR UTILIZÂND ÎNVĂȚAREA PRIN RECOMPENSĂ

Absolvent

Igescu Rareș-Andrei

Coordonator științific

Conf. Dr. Păduraru Ciprian

București, iunie 2026

1. Rezumat

Cuprins

1	Rezumat	1
	Cuprins	2
2	Introducere	4
2.1	Lorem ipsum	4
2.2	Dolor sit amet	4
3	Preliminarii	5
3.1	Sisteme de detecție bazate pe semnături (SIDS)	5
3.2	Sisteme de detecție bazate pe anomalii (AIDS)	6
3.3	Fundamentele Învățării prin Recompensă (RL)	7
3.3.1	Procese Decizionale Markov (MDP)	7
3.3.2	Ecuția Bellman și Deep Q-Networks (DQN)	8
3.4	Stadiul actual al cercetării	9
3.5	Obiectivele lucrării	10
4	Dezvoltarea Sistemului de Detecție	11
4.1	Pregătirea și preprocesarea datelor experimentale	11
4.2	Proiectarea mediului de antrenament	12
4.2.1	Spațiul stărilor și spațiul acțiunilor	12
4.2.2	Proiectarea funcției de recompensă	13
4.2.3	Episoadele de antrenament și generalizarea	14
4.3	Definirea și Antrenarea Agentului	14
4.3.1	Configurarea Parametrilor și a Politicii de Învățare	14
5	Integrarea și Testarea Sistemului în Timp Real	15
5.1	Implementarea Modulului de Captură a Traficului	15
5.1.1	Configurarea și Depanarea Senzorului	15
5.1.2	Generarea Datelor Live	15
6	Concluzii	16
7	Bibliografie	17

2. Introducere

2.1 Lorem ipsum

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam convallis dapibus ante in interdum. Aliquam vitae elit eu elit gravida facilisis.

2.2 Dolor sit amet

Fusce fermentum quis tellus vitae congue. Interdum et malesuada fames ac ante ipsum primis in faucibus.

3. Preliminarii

3.1 Sisteme de detecție bazate pe semnături (SIDS)

Sistemele de detecție bazate pe semnături reprezintă o metodă tradițională în securitatea rețelelor, funcționând pe un principiu similar programelor antivirus clasice. Aceste sisteme monitorizează traficul de rețea și compară conținutul pachetelor cu o bază de date predefinită de ”semnături” (reguli) asociate atacurilor cunoscute.

Din punct de vedere algoritmic, eficiența acestui proces depinde de viteza cu care se efectuează potrivirea modelelor (*pattern matching*). Pentru a face față volumului mare de date în timp real, majoritatea soluțiilor consacrate utilizează algoritmul **Aho-Corasick** [1]. Acesta construiește un automat finit determinist din baza de date de semnături, permițând căutarea simultană a tuturor modelelor într-o singură parcurgere a pachetului de date, garantând astfel o complexitate liniară $\mathcal{O}(n)$ care este totodată independentă de numărul de semnături (k).

Semnăturile menționate anterior reprezintă, practic, amprenta digitală a unui atac, definită printr-un șir specific de bytes. Cel mai reprezentativ exemplu industrial este **Snort**, care utilizează reguli de forma:

Dacă pachetul vine de la IP-ul X și conține șirul hexazecimal Y , atunci generează o alertă.

Limitări ale abordării bazate pe semnături

Deși sistemele bazate pe semnături oferă o rată scăzută de alarme false (*False Positives*) pentru alarmele cunoscute, rigiditatea acestora constituie o mare breșă în contextul contemporan al securității:

- **Incapacitatea de a detecta atacuri Zero-Day:** Deoarece funcționează exclusiv pe baza unei liste negre (*blacklist*), orice atac nou apărut, pentru care nu a fost încă scrisă o semnătură, va trece neobservat.
- **Vulnerabilitate la Polimorfism:** Atacatorii pot modifica ușor semnătura unui malware (prin criptare sau schimbarea unor biți nesemnificativi) fără a-i altera funcționalitatea malițioasă. Această tehnică face ca semnătura din baza de date să nu se mai potrivească exact, păcălind algoritmul de pattern matching.
- **Mentenanță costisitoare:** Baza de date necesită actualizări manuale și continue, proces adesea prea lent față de viteza de propagare a noilor vectori de atac.

Aceste limitări necesită tranziția către paradigme de detecție euristice și comportamentale, precum **Sistemele Bazate pe Anomalii (AIDS)**, capabile să generalizeze și să identifice intenții malițioase necunoscute anterior.

3.2 Sisteme de detecție bazate pe anomalii (AIDS)

Acest model, propus inițial de **Denning** [2], reprezintă un sistem expert universal pentru detecția intruziunilor în timp real, bazat pe ipoteza că abuzurile informatice pot fi identificate prin monitorizarea anomaliilor din jurnalele de activitate. Prin utilizarea profilurilor statistice și a metricilor de comportament, sistemul învață interacțiunile normale dintre utilizatori și resurse pentru a semnaliza devierile suspecte, oferind un cadru adaptabil oricărei platforme, indiferent de tipul vulnerabilităților sau al atacurilor.

Pentru a putea detecta cu succes traficul suspect sau malițios, sistemul trebuie mai întâi să învețe acțiuni normale. Astfel, procesul de funcționare al unui sistem bazat pe anomalii poate fi împărțit în două:

- **Faza de antrenare:** Sistemul analizează un volum mare de trafic (protocoale utilizate des, lățime de bandă) care poate fi considerat normal pentru a stabili un profil de referință (cunoscut drept *baseline*) al activității.
- **Faza de detecție:** Orice deviație de la acest standard va fi marcată de sistem ca anomalie și va ridica o alertă la nivelul traficului.

Avantajul major al acestor sisteme este capacitatea de a detecta atacuri de tip **Zero-Day** deoarece sistemul nu are nevoie să cunoască de dinainte amprenta atacului; este suficient ca traficul malițios să se comporte diferit față de cel normal.

Totuși, implementarea clasică a sistemelor AIDS vine cu o provocare, anume rata ridicată a alarmelor false (*False-Positive Rate*). Rețelele moderne sunt dinamice, cu trafic variabil, iar un comportament atipic, dar legitim (trimiterea bruscă a mai multor fișiere de dimensiuni mari) poate fi marcat ca trafic malițios.

Pentru a rezolva această problemă a alarmelor false și a crea un sistem capabil să se adapteze continuu la dinamica rețelei, soluțiile moderne recurg la algoritmi avansați de inteligență artificială (AI). Dintre aceștia, o paradigmă deosebit de promițătoare, care permite sistemului să învețe direct din consecințele deciziilor sale, este **Învățarea prin Recompensă (Reinforcement Learning)**.

3.3 Fundamentele Învățării prin Recompensă (RL)

Conform fundamentelor stabilite de **Sutton și Barto** [3], învățarea prin recompensă constă în capacitatea unui agent de a descoperi ce acțiuni trebuie întreprinse în anumite situații, cu scopul fundamental de a maximiza un semnal numeric de recompensă.

Multe sisteme reale nu au soluții perfecte, ci doar acțiuni mai bune în funcție de experiența acumulată până în acel moment. Un agent bazat pe învățarea prin recompensă încearcă să învețe cum să acționeze optim într-un mediu necunoscut. În contextul securității cibernetice, agentul joacă rolul sistemului de detecție, iar mediul din care se poate antrena este reprezentat de traficul rețelei monitorizate. Ciclul generic pentru un agent RL poate fi analizat în figura 3.1.

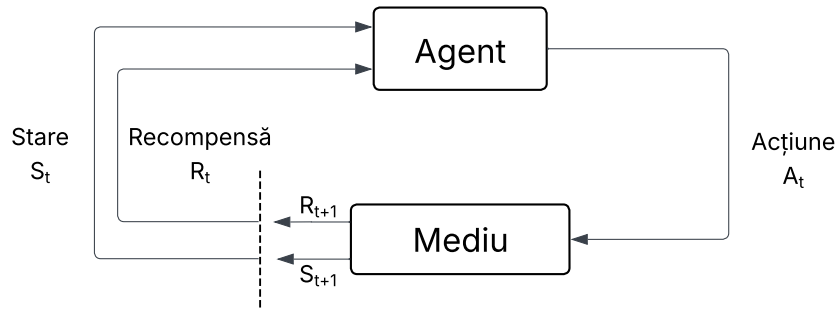


Figura 3.1: Ciclul de interacțiune Agent-Mediu în sistemul propus

3.3.1 Procese Decizionale Markov (MDP)

Procesele Markov sunt procese stochastice în care viitorul depinde doar de prezent. Mai precis, un proces Markov îndeplinește proprietatea Markov doar dacă probabilitatea de a ajunge într-o stare viitoare depinde exclusiv de starea curentă. Dacă spațiile de stări sunt finite, atunci acesta se numește proces de decizie Markov finit.

$$Pr(s_t | s_{t-1}, \dots, s_0) = Pr(s_t | s_{t-1})$$

Ecuția 3.1 Procese Markov de ordinul I

Această relație definește „independența de istoric”, sugerând că starea s_{t-1} încapsulează toate informațiile relevante din trecut necesare pentru a prezice starea viitoare s_t . În mod implicit, procesele Markov

se referă la procesele de ordinul I. Totuși, există situații complexe în care dependența se extinde asupra mai multor stări anterioare, ducând la procese de ordin superior:

$$Pr(s_t | s_{t-1}, \dots, s_0) = Pr(s_t | s_{t-1}, s_{t-2})$$

Ecuția 3.2 Procese Markov de ordinul II

Din puncte de vedere formal, un proces de decizie Markov este definit de următorul tuplu (S, A, P, R, γ) , unde:

- S reprezintă mulțimea finită de stări;
- A reprezintă mulțimea finită de acțiuni;
- $P(s' | s, a)$ este probabilitatea de a ajunge în starea s' după ce agentul alege acțiunea a în starea s ;
- $R(s, a, s')$ este recompensa primită pentru tranziția $(s \rightarrow s')$;
- $\gamma \in [0, 1]$ este factorul de discount, care controlează importanța viitorului.

În contextul detecției anomaliilor în trafic, proprietatea Markov presupune că pachetul curent (sau fereastra de trafic actuală) conține suficiente informații pentru a decide dacă este un atac, fără a fi necesară analiza întregului istoric al conexiunii de la inițializarea rețelei.

3.3.2 Ecuția Bellman și Deep Q-Networks (DQN)

Pentru a afla politica optimă, agentul analizează fiecare acțiune posibilă folosind funcția valoare-acțiune (*Action-Value*) $Q(s, a)$ care estimează recompensa așteptată pe termen lung. Actualizarea acestei funcții se face iterativ folosind Ecuția lui Bellman, care exprimă valoarea unei perechi stare-acțiune ca fiind suma dintre recompensa imediată și valoarea maximă estimată a stării următoare:

$$Q^*(s, a) = R(s, a, s') + \gamma \max_{a'} Q^*(s', a')$$

Ecuția 3.3 Ecuția Bellman

În varianta clasică, algoritmul Q-Learning memorează valorile lui Q într-un tabel. Această abordare a problemei funcționează atunci când spațiul stărilor este redus. Cu toate acestea, când vine vorba

de monitorizarea traficului unei rețele, numărul de stări este foarte vast, deoarece pachetele de date pot avea numeroase combinații de caracteristici. Astfel, utilizarea unei reprezentări tabelare este imposibilă în contextul prezentat.

Pentru a rezolva această problemă, **Mnih et al.** [4] au propus arhitectura Deep Q-Network (DQN). DQN elimină tabelul și utilizează o rețea neuronală profundă (cu ponderile θ) pentru a aproxima direct valorile optime: $Q(s, a; \theta) \approx Q^*(s, a)$.

În această abordare, caracteristicile traficului de rețea reprezintă datele de intrare (*input*) ale rețelei, iar ieșirile (*output*) sunt valorile Q estimate pentru acțiunile disponibile (Permite sau Blochează). Această generalizare permite sistemului IDS să identifice corect tipare de atac chiar și în configurații de trafic pe care nu le-a întâlnit în timpul antrenamentului.

3.4 Stadiul actual al cercetării

Utilizarea algoritmilor de Învățare prin Recompensă în securitatea cibernetică a cunoscut o creștere semnificativă, devenind o alternativă tot mai promițătoare la metodele clasice, cum ar fi Arborii de Decizie, care au nevoie de seturi de date corecte și complet etichetate și care totodată tind să își piardă eficiența în medii dinamice, unde tiparele de atac evoluează constant.

Studii relevante, cum sunt cele realizate de **Caminero et al.** [5] și **Lopez et al.** [6], au demonstrat că agenții RL pot fi utilizați cu succes pentru a detecta traficul malițios, modelând interacțiunile dintre atacator și sistemul de detectare a intruzinilor ca un joc secvențial. Această perspectivă permite adaptarea continuă a mecanismelor de detecție în funcție de comportamentul adversarului.

Totuși analiza literaturii actuale evidențiază două limitări majore:

1. **Utilizarea datelor învechite:** O mare parte din studiile existente încă își evaluează agenții de tip RL folosind seturi de date învechite, precum KDD99 sau NSL-KDD. Aceste informații nu mai reprezintă infrastructura modernă a unei rețele și nu includ tipologii actuale ale unui atac cibernetic, cum ar fi DDoS de tip Slowloris sau vulnerabilitățile specifice aplicațiilor web.
2. **Funcții de recompensă suboptimale:** Multe implementări se concentrează exclusiv pe maximizarea ratei de detecție (Recall), ignorând penalizarea adecvată a alarmelor false (False Positives), ceea ce face sistemele inaplicabile într-un mediu de producție real.

3.5 Obiectivele lucrării

Având în vedere limitările identificate în stadiul actual al subdomeniului, prezenta lucrare de licență își propune dezvoltarea, implementarea și evaluarea unui Sistem de Detecție a Intruziunilor (IDS) adaptiv, bazat pe o arhitectură Deep Q-Network (DQN).

Pentru a atinge acest scop principal, au fost stabilite următoarele obiective specifice:

- **O1. Modelarea mediului de rețea ca un Proces Decizional Markov (MDP):** Proiectarea unei funcții de recompensă personalizate care să penalizeze sever generarea de alarme false, forțând agentul să găsească un echilibru optim între securitate și disponibilitatea rețelei.
- **O2. Implementarea agentului RL:** Dezvoltarea arhitecturii DQN utilizând rețele neuronale profunde pentru aproximarea funcției de valoare, capabilă să proceseze spațiul continuu al caracteristicilor de trafic.
- **O3. Antrenarea și validarea pe date moderne:** Utilizarea setului de date **CICIDS2017** (Canadian Institute for Cybersecurity), care conține trafic benign actualizat și cele mai comune familii de atacuri moderne, asigurând astfel relevanța practică a sistemului.
- **O4. Analiza comparativă a performanței:** Evaluarea agentului RL în raport cu un model de referință bazat pe învățare supervizată (ex.: Random Forest), utilizând metrici specifice domeniului securității (Precizie, Recall, F1-Score).

4. Dezvoltarea Sistemului de Detecție

4.1 Pregătirea și preprocesarea datelor experimentale

Primul pas esențial în crearea sistemului inteligent de detecție a constat în organizarea setului de date. Pentru aceasta, a fost utilizat dataset-ul **CICIDS2017** [7], un standard contemporan pentru evaluarea sistemelor de securitate, care include atât trafic benign, cât și semnăturile celor mai frecvente atacuri cibernetice (Brute Force, DDoS, Atacuri Web, etc.).

Manipularea și curățarea volumului masiv de date s-au realizat utilizând biblioteca *Pandas* [8] din limbajul Python.

Etapă inițială de preprocesare a datelor a constat în eliminarea instanțelor invalide (de tip *Inf* și *NaN*) și în uniformizarea setului de date, demers esențial pentru a asigura convergența și stabilitatea procesului de antrenare a agentului. Pentru compatibilizarea datelor cu arhitectura unui model de decizie binară, eticheta primară a fost recodificată: traficul de rețea legitim a fost asociat cu valoarea **0** („Permite”), în timp ce toate instanțele de trafic malițios, indiferent de tipologia atacului, au fost agregate sub valoarea **1** („Blochează”). În faza finală, având în vedere discrepanțele majore de scală dintre parametrii de rețea (variind de la zeci de octeți la milioane de microsecunde), s-a impus aducerea tuturor variabilelor continue în intervalul standardizat $[0, 1]$. Această operațiune a fost implementată utilizând clasa *MinMaxScaler* din cadrul bibliotecii *scikit-learn* [9]. Etapa de normalizare previne riscul ca algoritmul de învățare să fie predominat de valorile absolute mari și se realizează prin aplicarea următoarei transformări matematice asupra fiecărei valori x dintr-o caracteristică:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Ecuția 4.1 Normalizarea datelor

Unde x_{min} și x_{max} reprezintă minimul, respectiv maximul caracteristicii respective în setul de date. Efectul vizual al acestei transformări este ilustrat în Figura 4.1, unde se poate observa cum distribuția originală a datelor este perfect conservată, însă transpusă integral în noul interval standardizat. Prin parcurgerea acestui flux, vectorul de stări obținut devine strict numeric și echilibrat dimensional, fiind complet pregătit pentru mediul de antrenament al agentului RL.

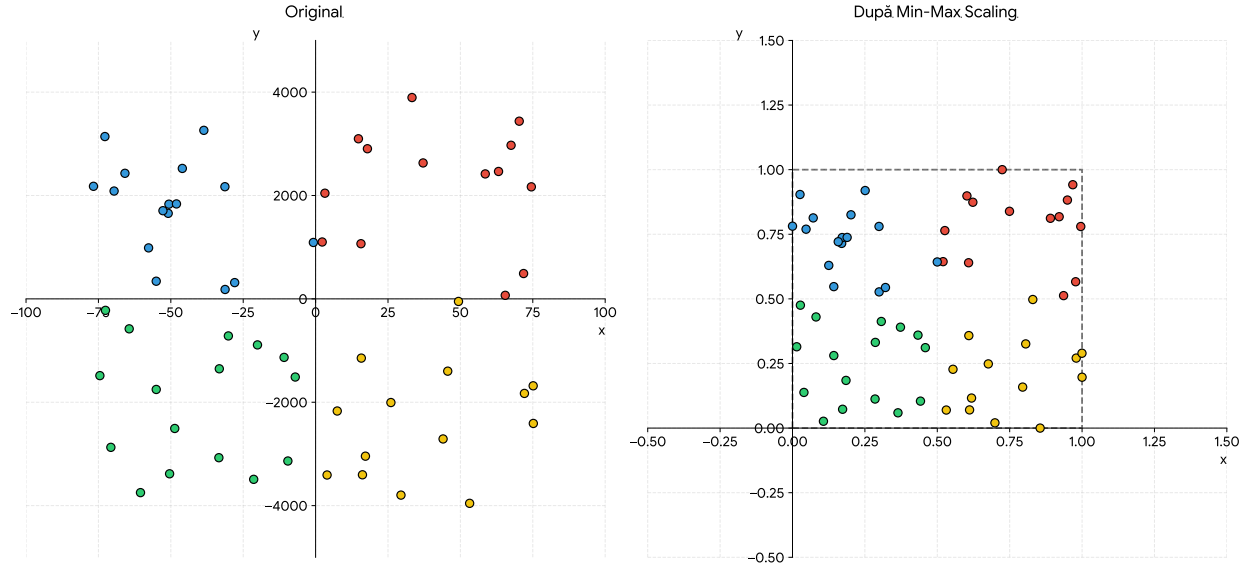


Figura 4.1: Exemplu de scalare a caracteristicilor

4.2 Proiectarea mediului de antrenament

Odată încheiată preprocesarea datelor, pasul următor a constat în crearea unei punți de comunicare între agentul decizional și datele de trafic. Astfel, a fost definit un mediu de învățare personalizat (*Custom Environment*), având ca punct de plecare structura standard oferită de biblioteca *Stable-Baselines3* [10] și respectând rigorile impuse de API-ul *Gymnasium* [11]. Prin adaptarea riguroasă a acestui cadru de lucru la specificul setului de date, problema detecției atacurilor de rețea a putut fi modelată conceptual sub forma unui Proces Decizional Markov (MDP).

4.2.1 Spațiul stărilor și spațiul acțiunilor

Pentru ca rețeaua neuronală să poată procesa informația, a fost necesară definirea limitelor mediului:

- **Spațiul stărilor:** A fost definit ca un spațiu continuu de tip *Box* (o zonă mărginită în spațiul n -dimensional). La fiecare pas t , starea s_t reprezintă un vector ce conține caracteristicile unui singur pachet de rețea. Datorită etapei anterioare de normalizare, s-a garantat că $s_t \in [0, 1]^N$, unde N este numărul de caracteristici (coloane) extrase din setul de date.
- **Spațiul acțiunilor:** Spațiul acțiunilor este discret, oferind agentului două decizii posibile la orice pas t : acțiunea $a_t = 0$ (permite pachetul) și acțiunea $a_t = 1$ (blochează pachetul).

4.2.2 Proiectarea funcției de recompensă

Nucleul mediului de antrenament, implementat în cadrul metodei *step()*, este reprezentat de funcția de recompensă $R(s_t, a_t)$. Aceasta a fost proiectată pentru a încuraja detecția precisă, dar și pentru a minimiza rata alarmelor false (un aspect critic în sistemele de producție). Astfel, recompensa acordată agentului este guvernată de următoarea funcție matematică definită pe ramuri:

$$R(s_t, a_t) = \begin{cases} 1, & \text{dacă } a_t = y_t \text{ (Clasificare corectă)} \\ -1, & \text{dacă } a_t = 0 \text{ și } y_t = 1 \text{ (Atac ratat)} \\ -2, & \text{dacă } a_t = 1 \text{ și } y_t = 0 \text{ (Alarmă falsă)} \end{cases}$$

Ecuția 4.2 Recompensa agentului pe ramuri

Unde y_t reprezintă eticheta reală (cunoscută de mediu) a pachetului analizat la pasul t . Penalizarea asimetrică (-2 pentru blocarea traficului legitim comparativ cu -1 pentru omiterea unui atac) forțează agentul să devină conservator, învățând să blocheze traficul doar atunci când certitudinea unui atac este foarte ridicată.

Algorithm 1 Logica de interacțiune și calculul recompensei

Require: $a_t \in \{0, 1\}$ ▷ Acțiunea primită de la agent (0 = Permite, 1 = Blochează)

- 1: $y_t \leftarrow$ eticheta reală a pachetului curent din setul de date
- 2: $r_t \leftarrow 0.0$ ▷ Inițializarea recompensei
- 3: **if** $a_t == y_t$ **then**
- 4: $r_t \leftarrow 1.0$ ▷ Agentul a clasificat corect
- 5: **else if** $a_t == 0$ **and** $y_t == 1$ **then**
- 6: $r_t \leftarrow -1.0$ ▷ Penalizare pentru atac ratat (Fals Negativ)
- 7: **else if** $a_t == 1$ **and** $y_t == 0$ **then**
- 8: $r_t \leftarrow -2.0$ ▷ Penalizare severă pentru alarmă falsă (Fals Pozitiv)
- 9: **end if**
- 10: $pas_curent \leftarrow pas_curent + 1$ ▷ Trecerea la următorul pachet de rețea
- 11: **if** $pas_curent \geq pasi_maximi$ **then**
- 12: $done \leftarrow \text{True}$ ▷ Episodul s-a încheiat
- 13: $obs_{t+1} \leftarrow \vec{0}$ ▷ Stare terminală (vector nul)
- 14: **else**
- 15: $done \leftarrow \text{False}$
- 16: $obs_{t+1} \leftarrow$ caracteristicile pachetului de la pas_curent
- 17: **end if**
- 18: **return** $obs_{t+1}, r_t, done$

4.2.3 Episoadele de antrenament și generalizarea

Pentru a preveni fenomenul de *overfitting* — situație în care agentul memorează secvența specifică a pachetelor din setul de date — antrenamentul a fost structurat în episoade a câte 1000 de pași. La începutul fiecărui episod, metoda *reset()* reinițializează starea sistemului alegând un index de start complet aleatoriu ($idx \in [0, \text{Total Pachete} - 1000]$). Această inițializare stocastică garantează că agentul este expus la instanțe de trafic diferite și variate la fiecare iterație, favorizând capacitatea acestuia de generalizare pe date noi.

4.3 Definirea și Antrenarea Agentului

Odată ce interacțiunea agentului cu mediul a fost configurată, am început dezvoltarea modelului în sine de Reinforcement Learning. Pentru această problemă s-a optat pentru algoritmul Deep Q-Network (DQN), recunoscut pentru eficiența sa în maparea stărilor complexe către acțiuni discrete cu ajutorul rețelelor neuronale adânci.

4.3.1 Configurarea Parametrilor și a Politicii de Învățare

Pentru a putea monitoriza și vizualiza performanța modelului în timpul procesului de antrenament, am encapsulat mediul prin intermediul clasei *Monitor*. Aceasta este o etapă crucială pentru generarea metricilor de evaluare, cum ar fi recompensa medie și durata episoadelor parcurse de agent. Arhitectura rețelei neuronale a fost stabilită prin utilizarea politicii *MlpPolicy* (Perceptron Multistrat). Această selecție este motivată de natura continuă a spațiului de observație (caracteristicile numerice ale pachetelor sunt normalizate în intervalul $[0, 1]$) și de un perceptron multistrat având capacitatea de a extrage eficient modele din aceste date multidimensionale.

Un element esențial în adaptarea algoritmului DQN pentru identificarea intruziunilor a fost ajustarea factorului de discount, simbolizat prin γ . În scenariile tradiționale de Reinforcement Learning, acțiunile agentului influențează starea viitoare a mediului. Cu toate acestea, în cazul analizei traficului, pachetele sosesc independent, iar decizia de clasificare a unui pachet nu afectează caracteristicile pachetului următor. De aceea, valoarea parametrului γ a fost stabilită la 0.0. Această modificare de arhitectură elimină zgomotul computațional, determinând agentul să se concentreze exclusiv pe maximizarea recompensei imediate (clasificarea corectă a pachetului actual), fără a încerca să prezică stări viitoare ce sunt, de fapt, complet independente.

— AICI VOI AVEA DE INTRODUS GRAFICELE DUPA CE LE FAC SA ARATE MAI BINE, DEOCAMDATA POT MERGE MAI DEPARTE SPRE ALTCEVA —

5. Integrarea și Testarea Sistemului în Timp Real

Acest capitol detaliază procesul de tranziție a modelului DQN antrenat către un mediu practic, capabil să analizeze traficul de rețea live. Sistemul a fost structurat în două componente independente: un senzor de captură a traficului și motorul de inferență bazat pe agentul DQN.

5.1 Implementarea Modulului de Captură a Traficului

Pentru a asigura compatibilitatea datelor live cu formatul setului CICIDS2017 pe care a fost antrenat agentul, s-a utilizat utilitarul CICFlowMeter portat în Python. Acesta are rolul de a intercepta traficul de rețea și de a extrage caracteristicile statistice ale fluxurilor de date.

5.1.1 Configurarea și Depanarea Senzorului

În cadrul implementării practice, utilitarul a fost configurat să asculte interfața de rețea activă a sistemului gazdă. Pe parcursul integrării, librăria originală a necesitat ajustări la nivel de cod sursă pentru a rula stabil pe sistemul de operare Windows.

Au fost identificate și corectate erori interne de tip `AttributeError` și `TypeError` în fișierele `sniffer.py` și `flow.py`, cauzate de gestionarea incorectă a listelor de argumente. După aplicarea acestor patch-uri în codul sursă al librăriei și instalarea driverului Npcap pentru interceptarea la nivel de pachet, senzorul a devenit complet funcțional.

5.1.2 Generarea Datelor Live

Modulul de captură monitorizează conexiunile și calculează cele peste 70 de caracteristici (precum durata fluxului, numărul de pachete, dimensiunea acestora) la încheierea fiecărei conversații de rețea. Aceste date sunt exportate în timp real într-un fișier CSV (`trafic_live.csv`), generând astfel vectorul de intrare exact pe care îl așteaptă rețeaua neuronală pentru evaluare.

6. Concluzii

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean dignissim metus justo, nec pharetra mauris tincidunt id.

7. Bibliografie

- [1] A. V. Aho and M. J. Corasick, “Efficient string matching: an aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [2] D. E. Denning, “An intrusion-detection model,” *IEEE Transactions on software engineering*, no. 2, pp. 222–232, 1987.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, second ed., 2018.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] G. Caminero, M. Lopez-Martin, and B. Carro, “Adversarial environment reinforcement learning algorithm for intrusion detection,” *Computer Networks*, vol. 159, pp. 96–109, 2019.
- [6] M. Lopez-Martin, B. Carro, and A. Sanchez-Esguevillas, “Network intrusion detection based on reinforcement learning and deep generative models,” *IEEE Access*, vol. 8, pp. 151474–151486, 2020.
- [7] Canadian Institute for Cybersecurity (CIC), “Intrusion detection evaluation dataset (cic-ids2017).” <https://www.unb.ca/cic/datasets/ids-2017.html>, 2017. Universitatea din New Brunswick (UNB). [Accesat la: 18.02.2026].
- [8] The pandas development team, “pandas-dev/pandas: Pandas.” <https://doi.org/10.5281/zenodo.18675244>, Februarie 2020. Versiunea 3.0.1. DOI: 10.5281/zenodo.18675244 [Accesat la: 25.02.2026].
- [9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

- [11] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.

8. **Anexe**

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean dignissim metus justo, nec pharetra mauris tincidunt id.